

MODEL CONTEXT PROTOCOL (MCP)

AGENT2AGENT PROTOCOL (A2A)

MULTI-AGENT SYSTEMS

Two Protocols for the Age of AI Agents

Francesca, Antonia, Paloma, Hanna

The N×M Integration Challenge

The Vertical Problem: Agent → Tools

Agents operate in continuous loops (ReAct pattern), calling tools and querying state. Without standardization, connecting **M** LLM agents to **N** internal databases and APIs requires . **M × N custom adapters**

- ⚠️ Bespoke auth, payload translation & error recovery per tool
- ⚠️ Context bloat: models receive fragmented, brittle schemas
- ✅ **Standardization:** Universal tool/resource interface (MCP)

The Horizontal Problem: Agent → Agent

Enterprises deploy specialized agents built on heterogeneous frameworks (LangGraph, CrewAI, PydanticAI, custom LLM loops). How can they delegate tasks across organizational boundaries?

- ⚠️ Incompatible memories, internal prompts, and execution models
- ⚠️ Proprietary IP leak if internal reasoning or weights are exposed
- ✅ **Standardization:** Opaque, delegated task collaboration (A2A)

Historical Parallels: HTTP, REST & LSP

HTTP & REST

Universal Resource Exchange

HTTP solved the transport layer (TCP streams), while REST provided uniform resource semantics (URLs + standard verbs GET/POST).

Turned M×N bespoke integrations into M+N standards.

Language Server (LSP)

Separation of IDE & Compiler

Microsoft's LSP enabled any editor (VS Code, Vim) to support any programming language through a standard JSON-RPC protocol.

Direct architectural inspiration for Anthropic's MCP design.

MCP & A2A Layer

Agent-Native Semantics

Neither reinvent transport: both ride on HTTP(S), JSON-RPC 2.0, and SSE. They add AI primitives: *Tools, Resources, Agent Cards, and Tasks*.

Standing on 30 years of web infrastructure rather than competing.

Why a Protocol — Not Just an API, a GUI, or XML

Manner	Scenario	Efficient	Broad Op. Range	Standardised	AI-Native
API	Server-to-server integration	✓	✗	✗	✗
GUI	Computer / mobile use	✗	✓	✓	✗
XML	Browser interactions	✗	✓	✗	✗
Protocol	Agent interaction	✓	✓	✓	✓

MCP: The "USB-C" of AI Context



Universal Port Metaphor

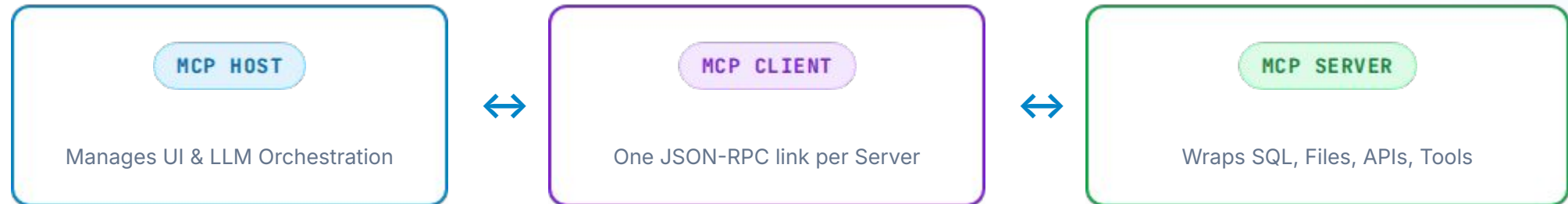
Just as USB-C replaced proprietary power cables, display ports, and data connectors with a single physical standard,

MCP establishes a universal logical interface connecting any AI application to any external tool or repository.

Core Protocol Tenets

- ✓ **Vendor Neutral:** Any model (Claude, GPT, Gemini, open weights); governed by the Linux Foundation since Dec 2025.
- ✓ **Introspectable:** Servers expose self-describing schemas via standard JSON Schema definitions.
- ✓ **Dual Transport:** Runs locally over [stdio](#) (IPC) or remotely over [Streamable HTTP](#).
- ✓ **Self-Describing Requests:** Since 2026-07-28, each request carries its own version and capabilities.

MCP Architecture & Topologies



1. The Host Container

The runtime environment holding the reasoning loop. If a host connects to 4 servers, it spawns 4 separate MCP Client instances.

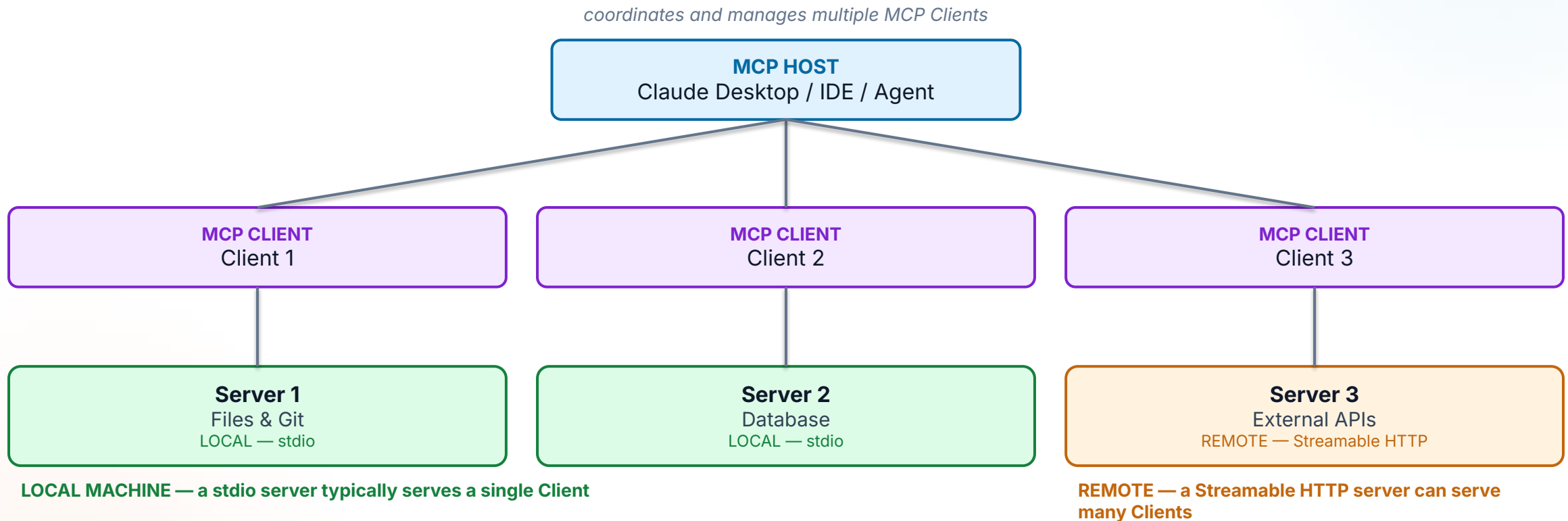
2. stdio Transport

Subprocess over standard input/output. No network port opened; ideal for local CLI tools and desktop apps.

3. Streamable HTTP Transport

Current remote transport (replaced HTTP+SSE). Stateless since 2026-07-28, so any server instance can answer and serve many Clients.

One Host, Many Clients — Local vs Remote Servers



The Request Cycle in Practice



1. Strategic Request

The Host analyzes the query with its LLM and, if connected to several Clients, decides which resources are relevant — then sends a context request to the right Client.

2. Executive Request

The Client, which holds the 1:1 connection to a given Server, builds and sends the concrete request: a Tool invocation, a Resource read, or a Prompt retrieval.

3. Response Phase

The Server operates on the resource and returns the result up the chain to the Client, then the Host — which combines everything into one coherent reply to the user.

MCP Primitives: Tools, Resources, Prompts

Tools

ACTION • MODEL-INVOKED

Executable functions returning data or state changes. The LLM receives the JSON schema and decides autonomously when to call it.

```
tools/list → schema  
tools/call {name, args}
```

Resources

DATA • READ-ONLY

Static or dynamic context data identified by URIs (e.g., **db://catalog/items** or file content). Read without executing code.

```
resources/list → URIs  
resources/read {uri}
```

Prompts

WORKFLOW • USER-INVOKED

Pre-packaged instruction templates exposed by the server. Often triggered via slash commands in chat clients or agent pipelines.

```
prompts/list → templates  
prompts/get {name, args}
```

Inside an MCP Server: Python FastMCP

```
catalog_server/server.py FastMCP SDK

from mcp.server.fastmcp import FastMCP
import sqlite3

mcp = FastMCP("Product Catalog")

# Automatic schema derivation via type hints
@mcp.tool()
def query_products(max_price: float = 100.0) -> list[dict]:
    """Search product catalog filtering by price."""
    conn = sqlite3.connect("catalog.db")
    cursor = conn.cursor()
    cursor.execute(
        "SELECT id, name, price FROM products WHERE price <= ?",
        (max_price,)
    )
    rows = cursor.fetchall()
    return [{"id": r[0], "name": r[1], "price": r[2]} for r in rows

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Key Implementation Highlights

- 1. Zero Manual JSON Schemas:** Parameter types (`float`) and docstrings are introspected at runtime (via Pydantic) to build each Tool's JSON Schema.
- 2. Encapsulated Execution:** The connection and SQL stay inside the server. The LLM only passes a typed argument, bound as a query parameter.
- 3. Swappable Transport:** Changing `stdio` to `Streamable HTTP` exposes the tool as a microservice without touching function logic.

Tool Context & Attention Budgets



The Attention Budget

LLM attention is finite. Loading dozens of tool schemas creates *context pollution*, degrading reasoning accuracy and causing tool mis-selection.

"If a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better."



Dynamic Pruning

Rather than dumping 50 tools into system context at launch, advanced hosts utilize **Just-in-Time (JIT) tool activation** based on user intent.

- ✓ Semantic routing of relevant tools
- ✓ Partitioning into task-specific sub-servers



Granularity Trade-off

Fine-grained:

20 atomic tools → high schema overhead & token consumption.

Coarse-grained:

1 generic tool (e.g. **fhir-request**) → shifts semantic burden into runtime reasoning.

Data Privacy Levers in MCP

The five architectural levers

Server as mediator: filters data before it reaches the model.

Local transport (stdio): no port opened, but results still reach the model: with a cloud LLM, data leaves the machine.

Least privilege: `get_summary(id)` instead of `execute_sql(query)`.

Per-user authentication: permissions propagated via OAuth from Client to Server.

Traceability (auditability): every call is a logged JSON-RPC message with explicit parameters.

Protocol ≠ Compliance

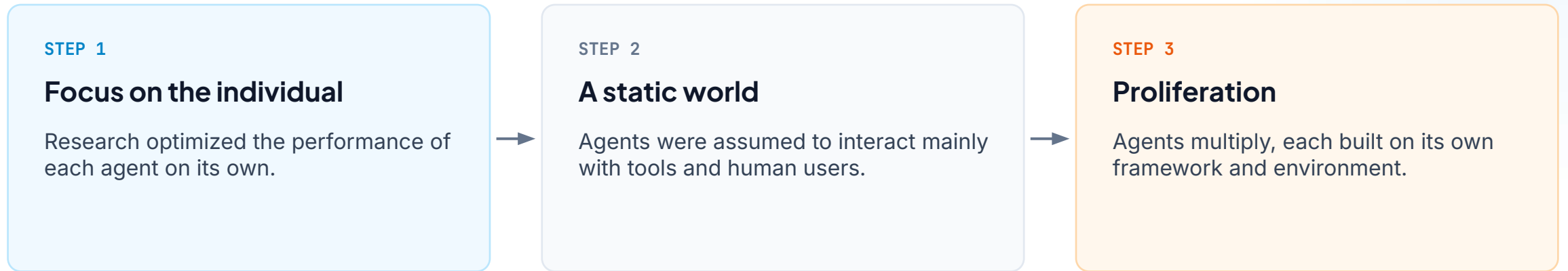
None of this is automatic. Implementing MCP correctly does not make a system HIPAA- or GDPR-compliant by itself.

Compliance requires configuring these five levers on purpose — plus the usual organizational controls: contracts, encryption, retention policies.

MCP Compared to a Traditional REST API

Criterion	REST API	MCP
Tool discovery	Out-of-band: docs or an OpenAPI file, read by developers	In-band at runtime (tools/list), read by the model
Typing	Implicit, or via OpenAPI	Native JSON Schema per Tool
Connection state	Stateless: each call is independent	Also stateless since spec 2026-07-28: no handshake, no session; each request carries version & capabilities
LLM integration	Ad hoc, via custom wrappers	Native — designed for it
Security	OAuth scopes or API keys; granularity is a design choice	One standard OAuth 2.1 flow any Host can follow; per-Tool scoping is still a server design choice

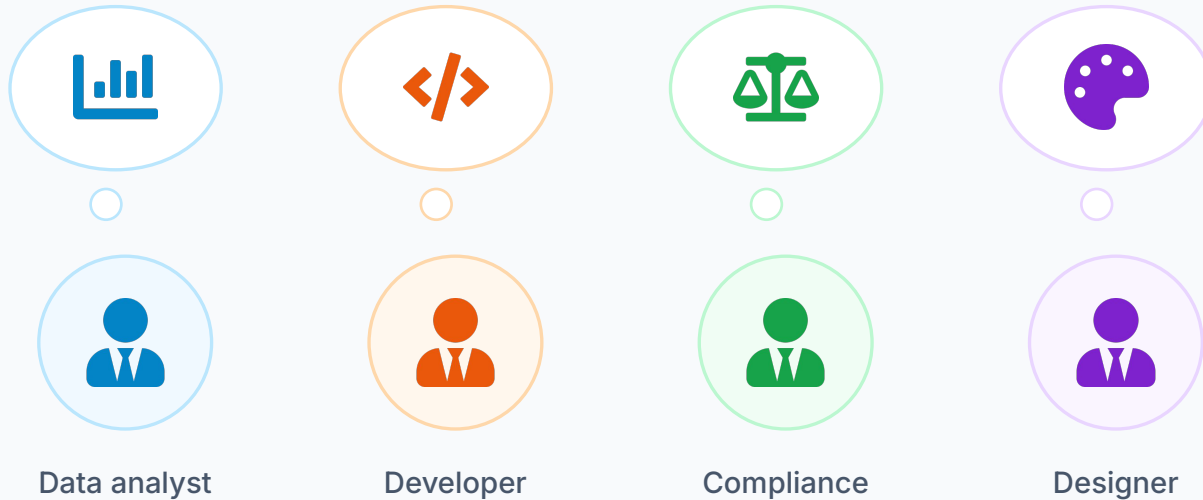
From Better Agents to Isolated Agents



“As these agents proliferate, they increasingly operate in silos, limited by the proprietary frameworks and local environments in which they were conceived.”

Silo: an agent that works well on its own but cannot talk to anything outside its framework.

A Team of Specialists That Never Talks



Different frameworks • different formats • no shared channel

Each specialist is excellent, and each understands the product in their own way.

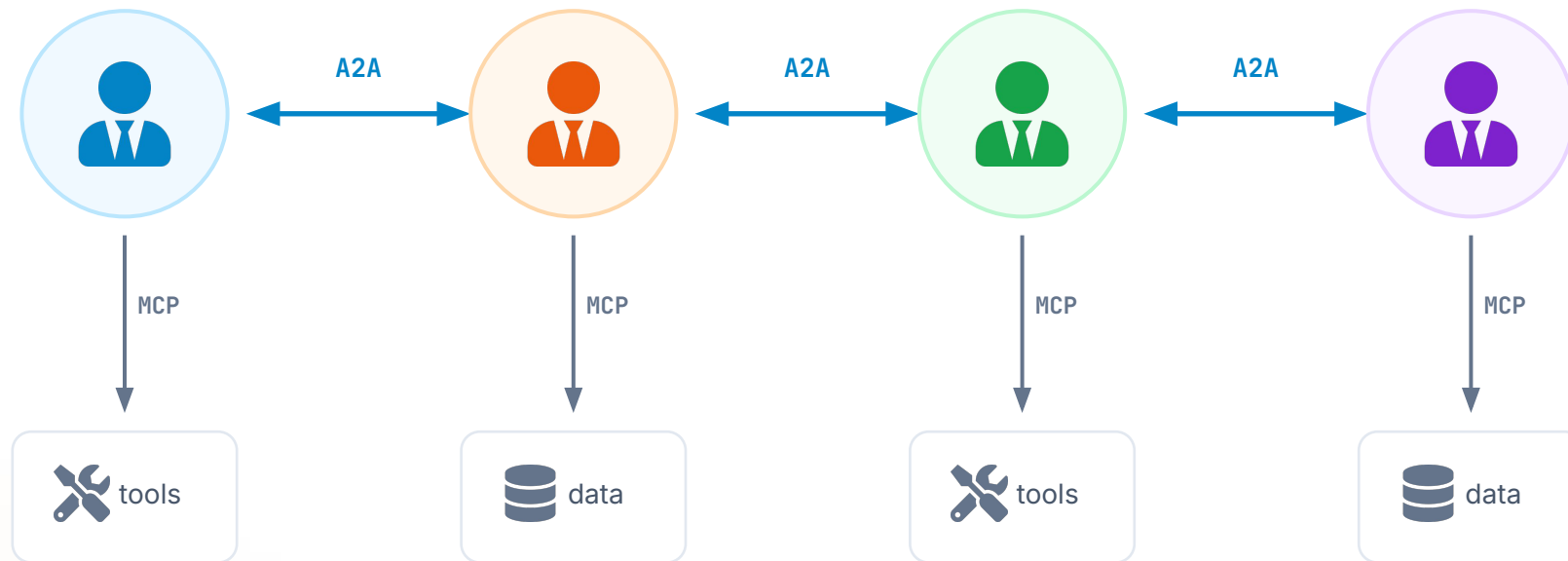
Inconsistent results

Each agent answers from its own partial view; outputs conflict.

Wasted potential

Specialists cannot combine skills, so complex cross-domain tasks stay out of reach.

Tools Are Not Enough: Agents Must Talk



MCP • VERTICAL

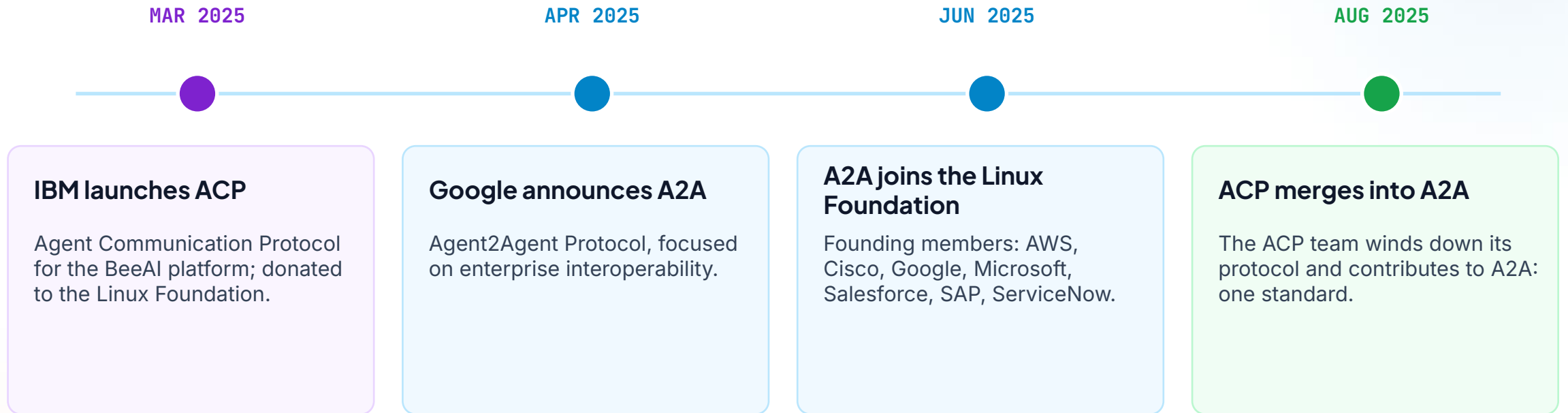
Each agent reaches its own tools and data.

A2A • HORIZONTAL

Agents discover each other and delegate tasks.

An isolated agent with tools is still a single silo. Same specialists, same colors (each keeps its own framework), now connected.

One Open Standard: Origin & Governance



Why neutral governance? A protocol between competing companies only works if no single company owns it. Otherwise, the agentic web would have an owner.

Key Design Principles



Simplicity

Reuses existing standards instead of reinventing core technology.

HTTP • JSON-RPC • SSE



Enterprise readiness

Standard web practices for authentication, authorization, privacy, tracing and monitoring.

AUTH • TRACING



Asynchronous

Native long-running tasks, even when agents or users disconnect.

STREAM • PUSH



Modality independent

Many content types, beyond plain text.

TEXT • FILES • JSON

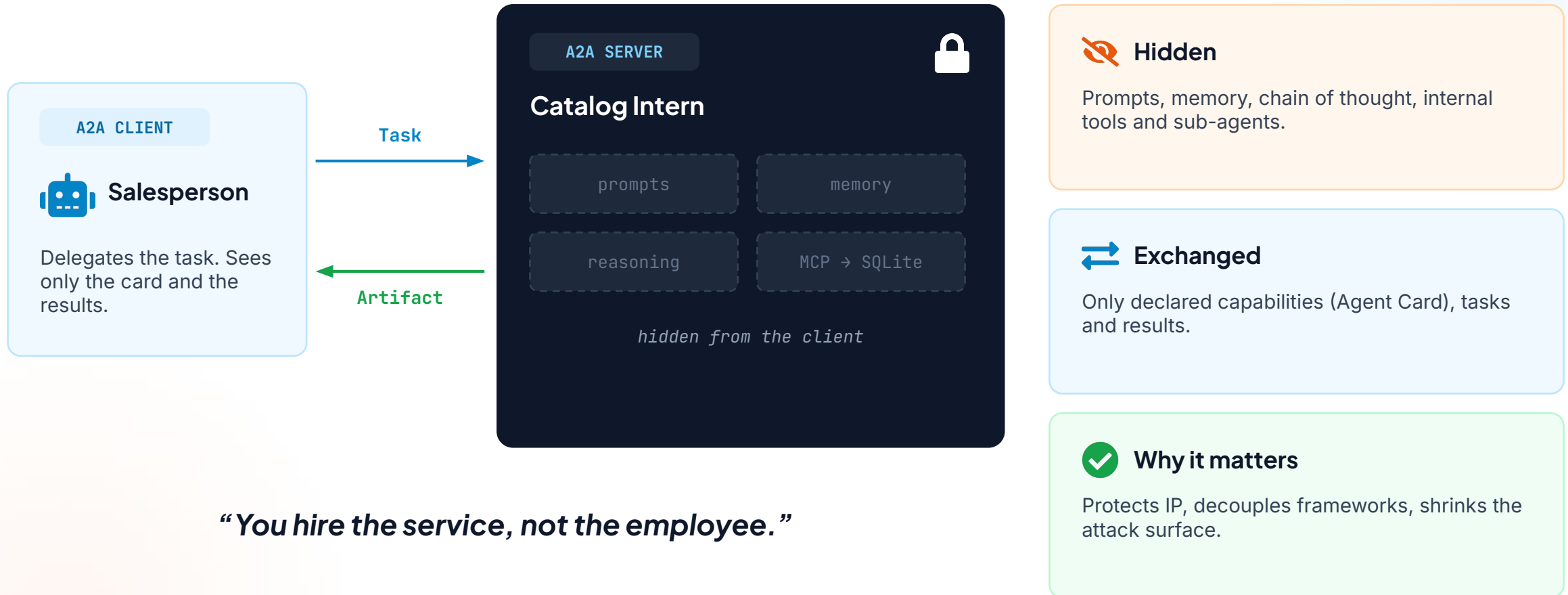


Opaque execution

Collaborate without exposing internal logic, memory or tools.

NEXT SLIDE →

The “Opaque Black Box” Principle



“You hire the service, not the employee.”

The Agent Card: A Digital Business Card

 /.well-known/agent-card.json

```
{
  "name": "Enterprise Catalog Intern",
  "description": "Queries products & inventory",
  "url": "https://agents.corp/intern",
  "version": "1.0.0",
  "capabilities": { "streaming": true },
  "skills": [{
    "id": "search-products",
    "name": "Product Search by Budget",
    "examples": ["find products under 20 euro"]
  }],
  "authentication": { "schemes": ["Bearer"] }
}
```

Published at a well-known URI, so any client can discover the agent and decide if it fits the task.



1. Skill registry

skills

Skill IDs and example prompts that the caller's LLM uses to match tasks.



2. Capability negotiation

capabilities

Declares support for streaming (SSE) or push notifications.

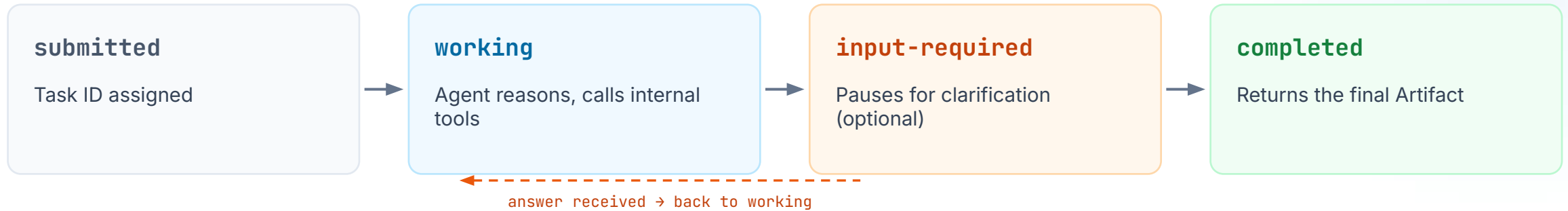


3. Security contract

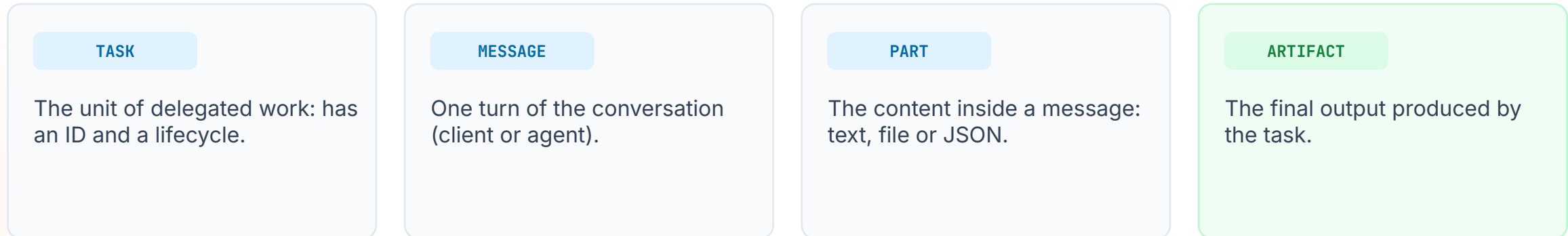
authentication

Declares the required authentication (e.g., OAuth, mTLS, bearer tokens).

Task, Message, Part & Artifact



Terminal states & recovery: **completed** · **failed** · **canceled** Every task ends explicitly, with structured error diagnostics.



Transport: Sync, Stream & Push



Request / Response

JSON-RPC over HTTPS. The client sends a message; the server returns the task state or a fast answer.

BEST FOR QUICK QUERIES



Server-Sent Events

One open connection streams progress: working → partial artifact → completed.

BEST FOR LIVE PROGRESS

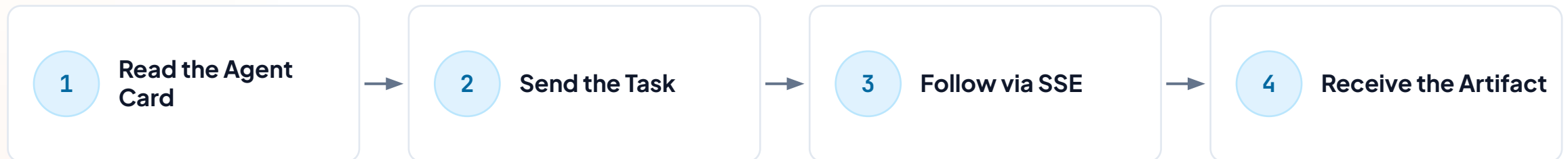


Push notifications

The client registers a webhook; the server calls back when a multi-hour task finishes.

BEST FOR LONG TASKS

END-TO-END EXCHANGE



What A2A Does Not Yet Solve



Capability verification

Can the agent really do what it claims?

Today: the Agent Card is self-declared.



Economic negotiation (SLA)

Cost, deadline, quality, penalties.

Today: no standard way to agree on terms.



Global discovery

Finding agents you don't know yet.

Today: you must already know the agent's URL.



Decentralized identity

Proving who you are without a central issuer.

Today: API keys, OAuth, mTLS.

A2A standardizes **how to ask**, not **whom to trust** or **what it costs**.

Proposed Architecture: A Layered Model

RESEARCH PROPOSAL • PREPRINT 2026

THREE PRINCIPLES

Semantic transparency

The meaning of each request is explicit. It does not contradict the black box: logic stays hidden.

Decentralized governance

No single authority controls identity or registry.

Transport agnosticism

The same messages over any transport.

L4 Governance & Security

Zero-trust: DIDs, Verifiable Credentials, a digital signature on every message.

L3 Negotiation

Agent Card exchange and dynamic SLAs: scope, resources, cost, error handling.

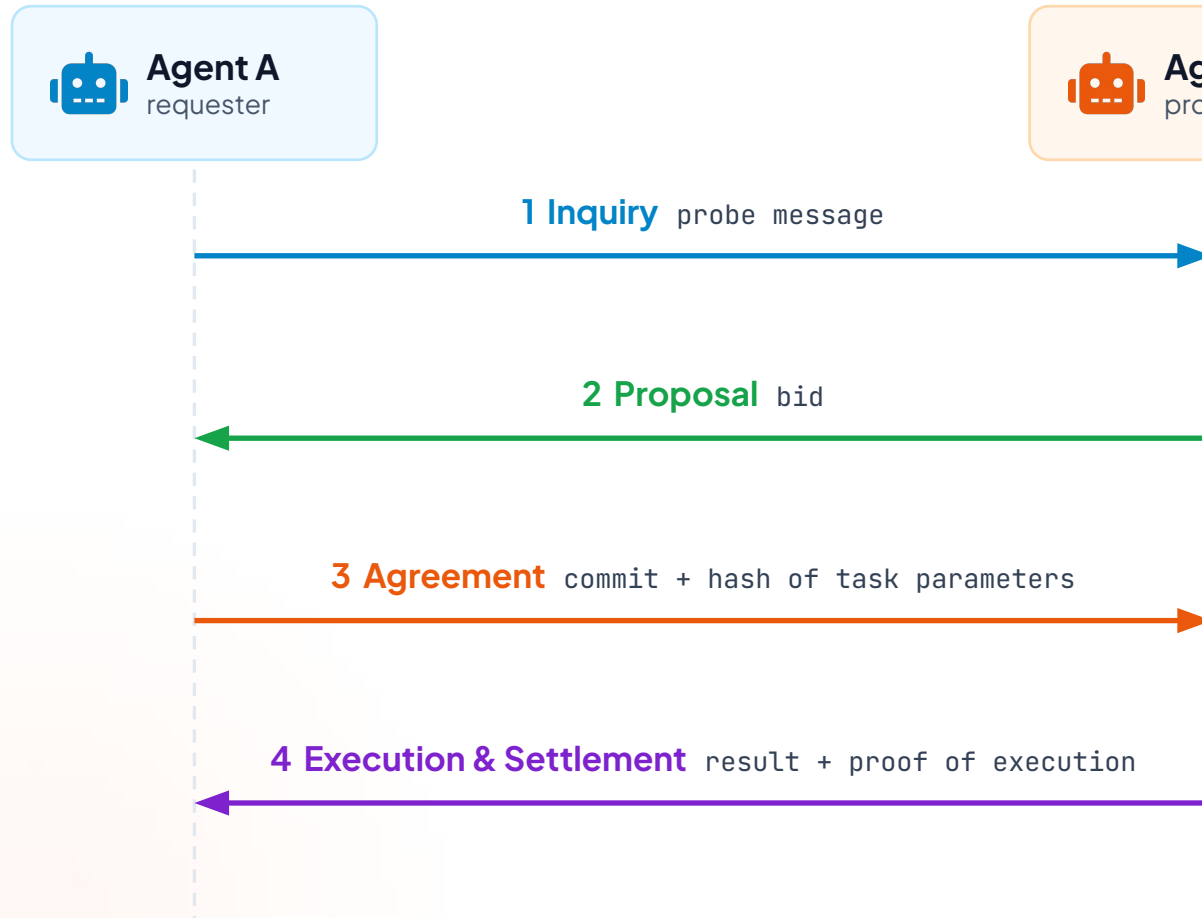
L2 Semantic ★ most critical

Intent ontology: goals translated into JSON-LD, a universal schema for agent actions.

L1 Transport

Connections between agents: packet delivery, sessions, basic encryption.

The Negotiation Lifecycle



 **All four steps are mandatory**

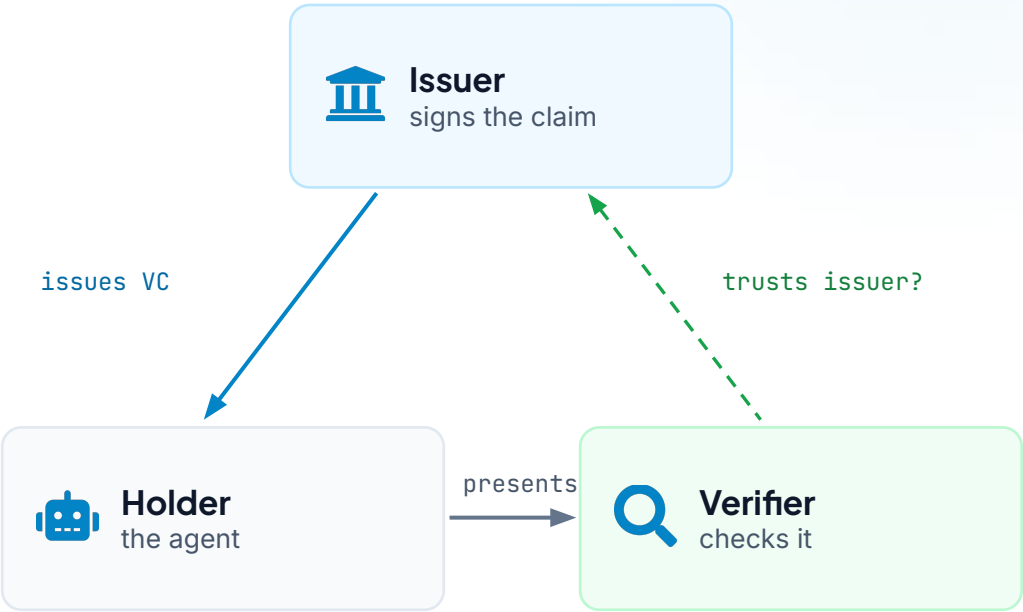
No task is executed without an agreement on the terms of engagement.

Identity & Trust: DIDs and Verifiable Credentials

AUTHENTICATION • WHO ARE YOU?

	API key	DID
Created by	The service issues it	The agent itself
Mechanism	Shared secret on every request	Key pair: sign with private key
If leaked	Anyone can reuse it	Private key never travels
Scope	One service	Portable across systems
Central authority	Yes	No

AUTHORIZATION • WHAT MAY YOU DO?

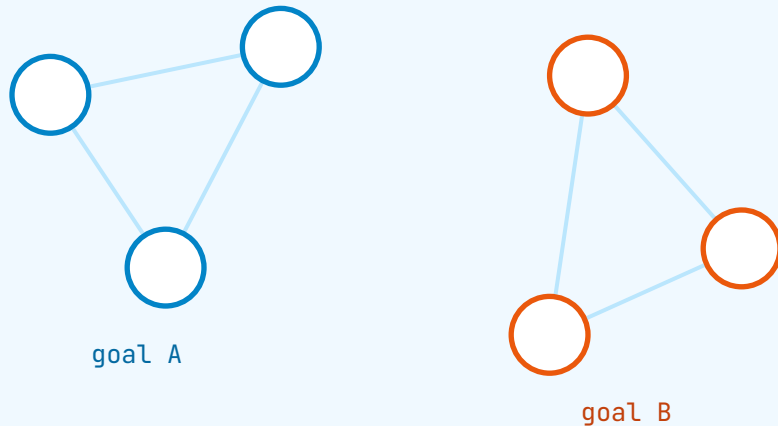


Claiming to be a bank is not enough: the credential must be signed by an issuer the verifier already trusts. VCs do not eliminate trust; they move it to a few trusted issuers.

Federated Orchestration & Hybrid Discovery

Federated orchestration

No central or master agent. Agents dynamically form and dissolve coalitions around a specific goal.



HYBRID DISCOVERY MODEL

Local discovery

Inside an enterprise network, agents find each other with a broadcast-based protocol.

Global discovery

Across platforms: a distributed hash table (DHT) backed by a consortium blockchain, so agent records are immutable, transparent and globally accessible.

Takeaways



A2A today

Standardizes discovery (Agent Card) and delegation (Task) between opaque agents, on HTTP, JSON-RPC and SSE.



Research frontier

Targets trust (DIDs, VCs), negotiation (SLAs) and global discovery (DHT + blockchain).



Open questions

Complexity and latency; a single-author preprint without independent validation.

Next → MCP vs. A2A: how the two protocols compare and work together in practice.

MCP vs. A2A: Side by Side

Dimension	Model Context Protocol (MCP)	Agent2Agent Protocol (A2A)
Connects	An agent to tools, data and prompts (vertical)	An agent to another agent (horizontal)
Who decides how	The caller’s LLM picks the tool and its arguments	The remote agent plans and executes on its own
Unit of work	Tool call (tools/call); tasks via an official extension	Task with a lifecycle, or a direct Message
What stays hidden	The implementation; the interface is fully typed	Prompts, memory, reasoning and internal tools
Discovery	tools/list at runtime (plus server/discover)	Agent Card at /.well-known/agent-card.json
Transport	stdio or Streamable HTTP; stateless since 2026-07	JSON-RPC, gRPC or REST over HTTPS; SSE, webhooks
Governance	Anthropic (2024) → Agentic AI Foundation, Linux Foundation (Dec 2025)	Google (Apr 2025) → Linux Foundation (Jun 2025)

The Official Position: Use Both



"MCP is vertical. It deepens a single agent. [...] A2A is horizontal. It connects agents across that boundary."

"Used together, MCP gives each agent depth, and A2A gives your system reach."

A2A PROTOCOL DOCUMENTATION, "A2A AND MCP"



Why not MCP for everything?

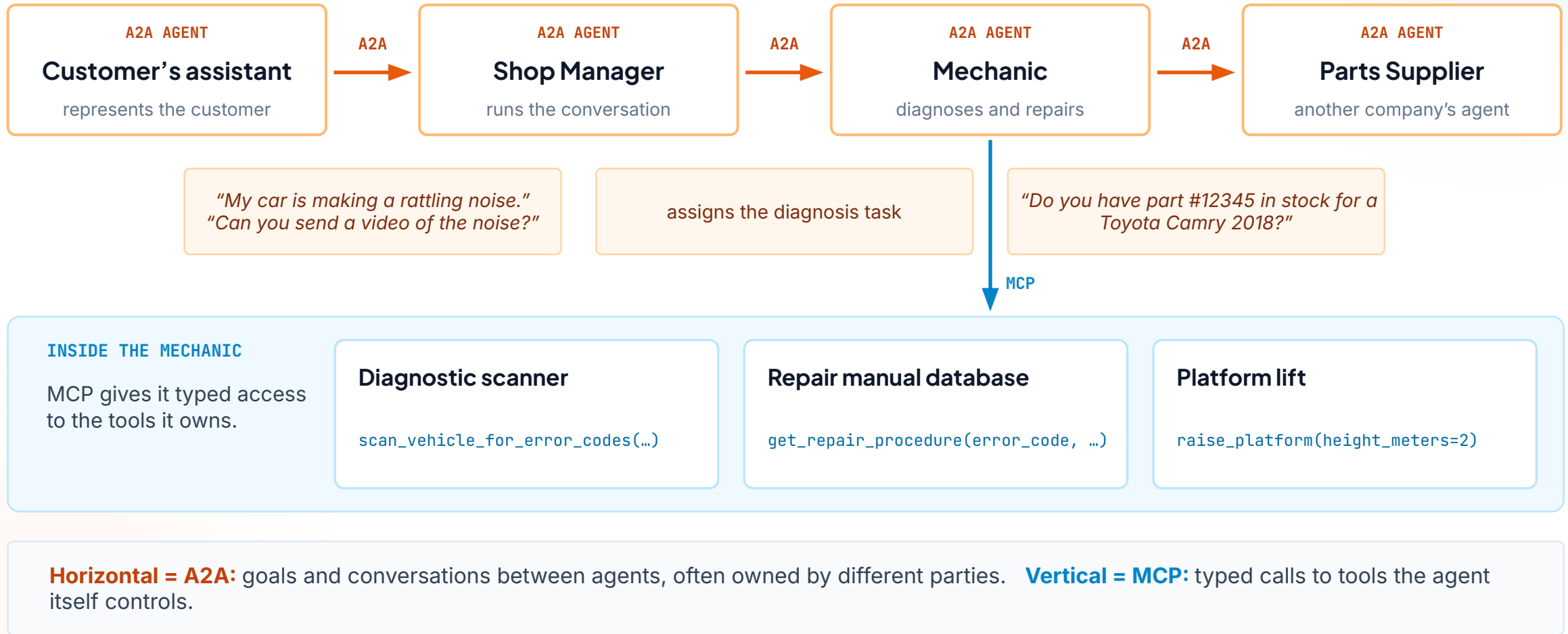
Wrapping an agent as an MCP tool ("agent-as-tool") is common, and it works for short, single-turn sub-tasks inside your own trust boundary. It breaks down when the callee must plan, ask back, refuse, or belongs to another organization.



Why not A2A for everything?

Putting a SQL query or a calculator behind an Agent Card and a task lifecycle adds discovery, state and negotiation overhead to what is essentially one function call. For tools, MCP is lighter and more direct.

Putting It Together: The Auto Repair Shop



The Line Is Blurring: 2025 to 2026



MCP → REST Stateless requests and header-based routing. Critics ask whether MCP is “just an API again”.

MCP → A2A Durable tasks and mid-call input: the very mechanics A2A was built around.

The mechanics are converging. The roles are not.

The Debate: Is Stateless MCP Just an API?

a tool call on the wire, spec 2026-07-28

POST /mcp HTTP/1.1

Mcp-Method: tools/call

Mcp-Name: search

Content-Type: application/json

```
{ "jsonrpc": "2.0", "id": 7,  
  "method": "tools/call",  
  "params": { "name": "search", ... } }
```

WHAT CHANGED

No handshake, no session. Each request stands alone, so load balancers, gateways and rate limiters can route tool calls by header, without parsing JSON.

THE SKEPTICS

"we invented a stateful protocol, discovered state is hard to scale, stripped it out, and arrived at 'just send a POST request.'"

Hacker News comment, quoted in InfoQ

THE DEFENSE

"The core advantage MCP gave you was a standard that by virtue of being blessed by AI providers people had strong incentives to actually implement."

Hacker News comment, quoted in InfoQ

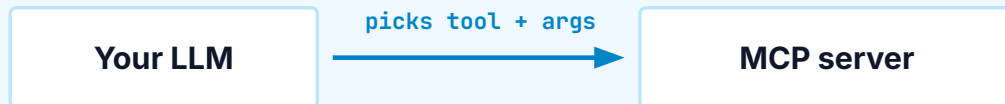
OUR READING

The transport converged on purpose. What MCP standardizes is the contract: runtime discovery, schemas written for the model, one auth flow, and a client every host implements.

What Still Separates Them: Who Is in Control



MCP: the caller decides



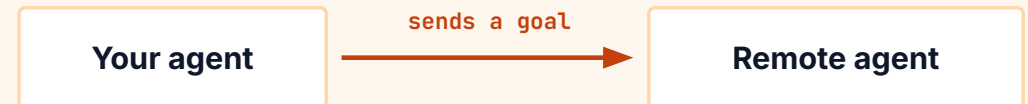
Choice: your LLM picks the tool and fills in its arguments.

Server: executes a capability; it has no goals of its own.

Trust question: is this tool safe to hand to my model?



A2A: the callee decides



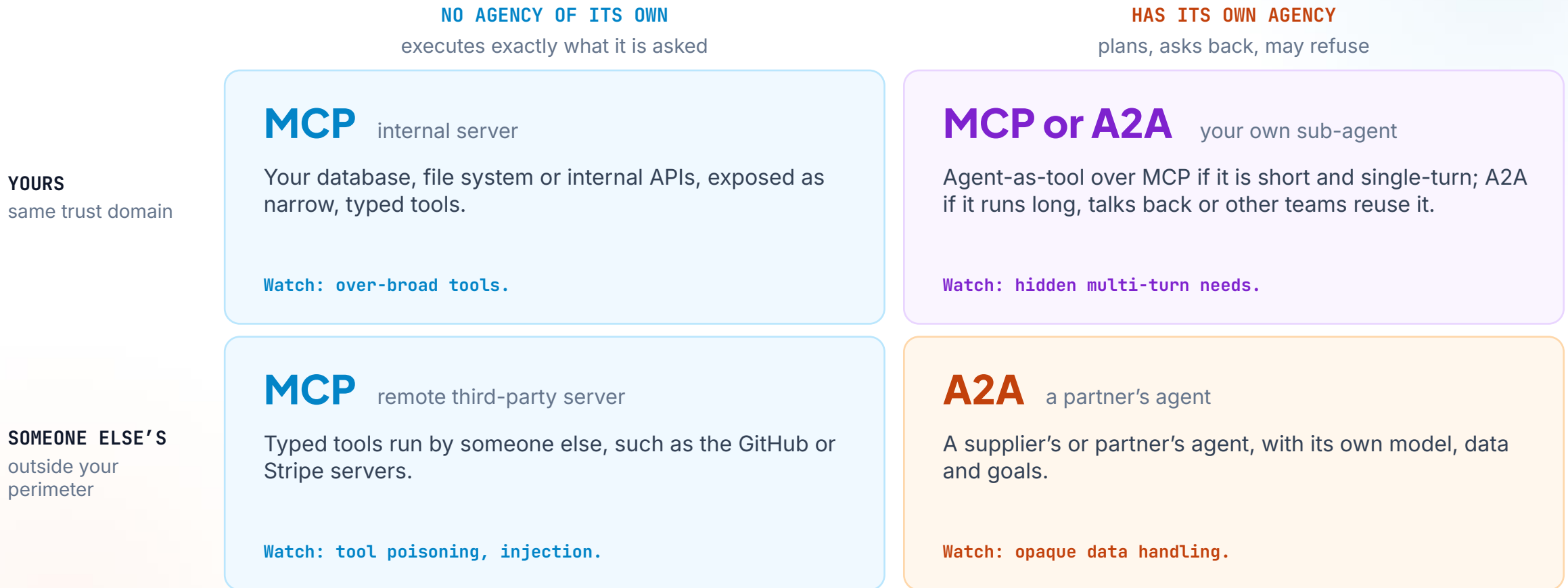
Choice: you send a goal; the remote agent plans how to reach it.

Agent: may ask back (input-required) or decline (rejected).

Trust question: will this other party handle my data well?

“A2A is about agents partnering on tasks; MCP is more about agents using capabilities.” [A2A DOCS](#)

MCP, A2A, or Both? A Two-Question Guide



Agency picks the protocol. Ownership decides how much you must trust it.

Trust Boundaries & Data Privacy



MCP: What Can Your Model Reach?

Not automatically internal: remote, third-party servers sit outside your perimeter.

Known attacks: tool poisoning (instructions hidden in tool descriptions) and prompt injection via tool results.

stdio ≠ no egress: the channel stays local, but results enter the model's context; with a cloud LLM, data leaves.

Levers: server-side filtering, least-privilege tools, per-user OAuth, logged calls.



A2A: Trusting an Opaque Partner

Invisible by design: you cannot see how the remote agent stores or uses your data.

Minimize: send only the fields the skill needs, never the whole record.

Declared auth: schemes in the Agent Card; a signed card proves who published it, not how it behaves.

Contracts: data processing agreements and vendor due diligence, not just protocol features.



With MCP you mostly protect your data from your own agent: a configuration problem. **With A2A you extend trust to someone else's agent:** a governance problem.

What Has Already Gone Wrong



MCP

INVARIANT LABS • APR 2025

Tool poisoning

A malicious server hides instructions in a tool description the user never sees. In the demo, an innocent-looking add tool made Cursor leak `~/.cursor/mcp.json` and SSH keys.

Variant: “rug pull”, the description changes after approval.



MCP

INVARIANT LABS • MAY 2025

Injection via tool results

A malicious issue in a public repo hijacked an agent (Claude 4 Opus in Claude Desktop) using the official GitHub MCP server. It copied private-repo data into a public pull request.

No server bug: the agent trusted text it read.



A2A

PALO ALTO UNIT 42 • OCT 2025

Agent session smuggling

A malicious remote agent used a multi-turn session to slip extra instructions to the client agent: it extracted system instructions and tool schemas, and triggered an unauthorized stock trade.

Not a protocol flaw: it abuses implicit trust.

Common thread: the protocol worked as designed; the model trusted text it should not have. Mitigations: least privilege per session, pinned and scanned tool descriptions, out-of-band confirmation for sensitive actions, signed Agent Cards.

A Standard Interface Is Not a Capable Agent

MCP-UNIVERSE • SALESFORCE AI RESEARCH

43.7%

best success rate on real MCP servers (GPT-5)

6 domains and 11 real servers: navigation, repositories, finance, 3D design, browser automation, web search. Grok-4 reached 33.3%, Claude-4.0-Sonnet 29.4%.

FHIR-AGENTBENCH • CLINICAL QA

50.0%

best answer correctness on clinical FHIR data

2,931 real clinical questions. Best agent: o4-mini, multi-turn, with a retriever and a code interpreter. Typical errors: wrong resource type, broken references between nested resources.

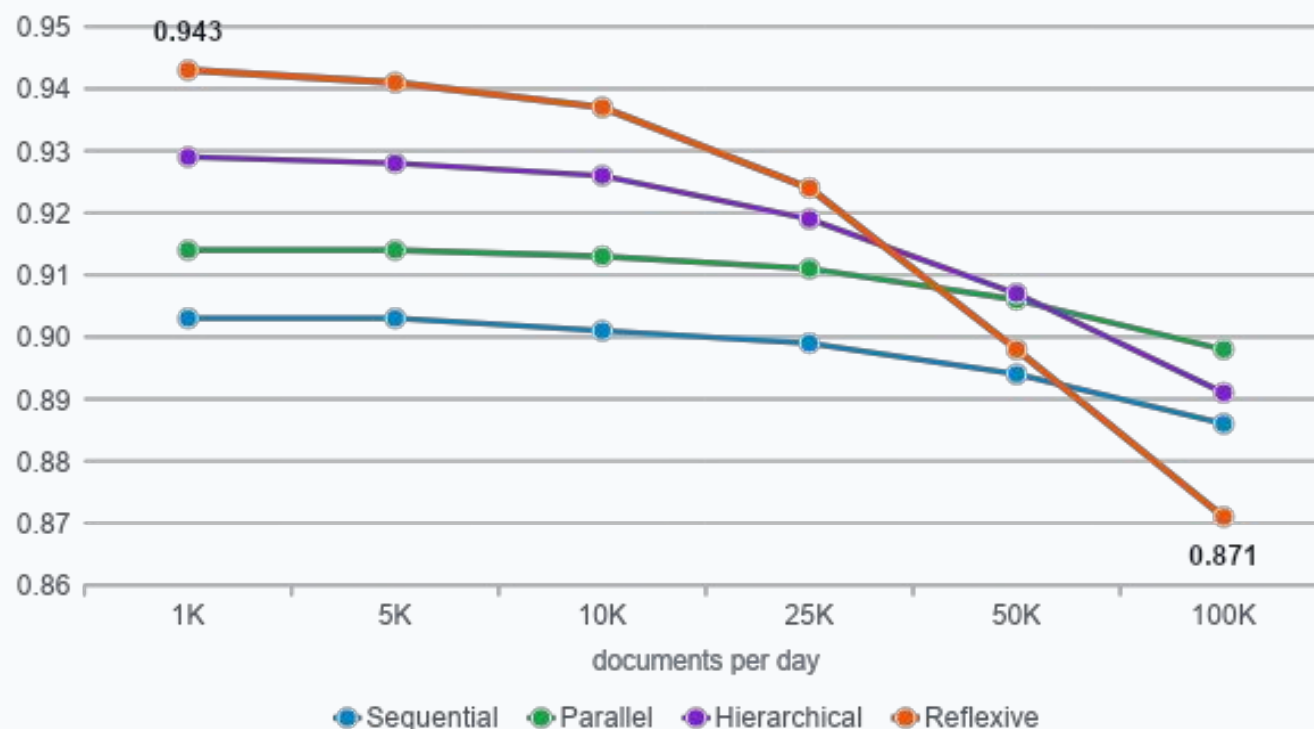


Neither failure is a protocol failure. MCP and A2A standardize how agents connect, not how well they reason: tool design, context engineering and evaluation remain the bottleneck.

Orchestration Has a Price

Field-level F1 as daily volume grows

10,000 SEC filings • Claude 3.5 Sonnet • fixed 8-node cluster



0.943 F1 at 2.3× cost

Reflexive self-correcting loop: most accurate at low volume, and the most expensive.

1.4× cost

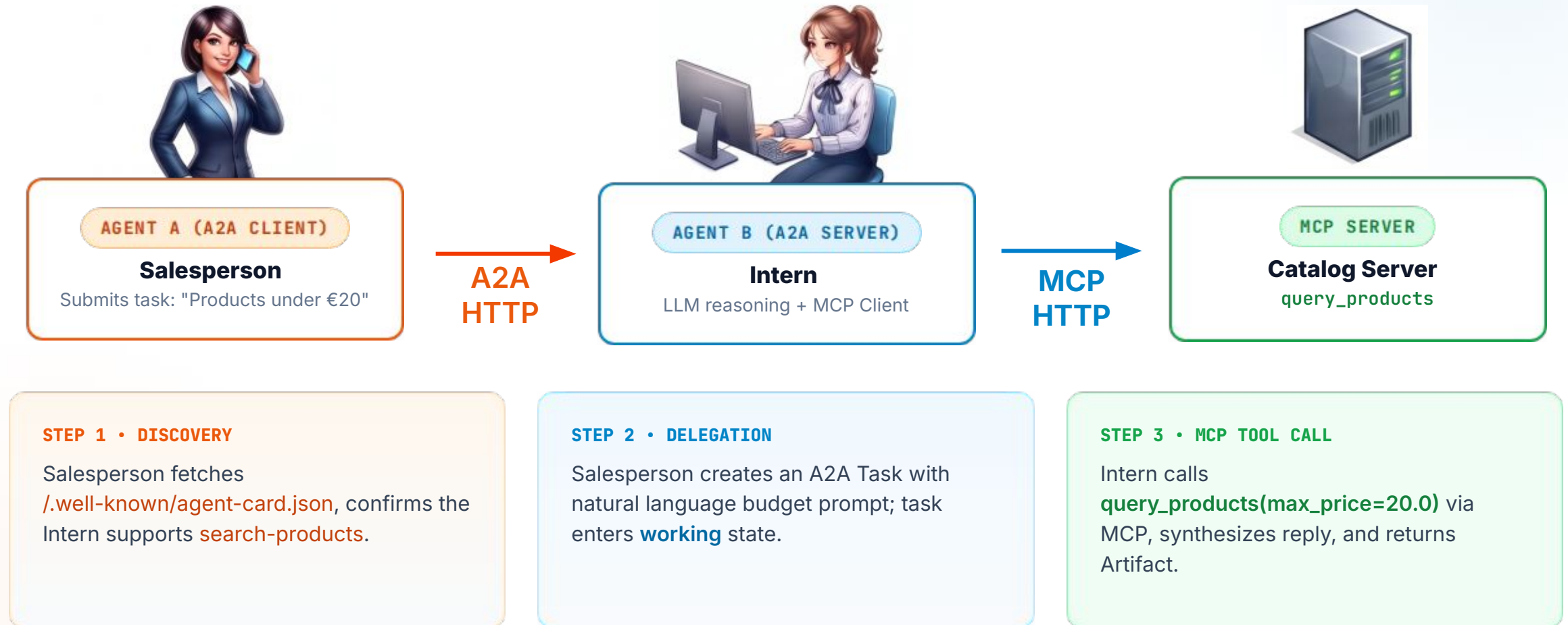
Hierarchical supervisor-workers: the best cost-accuracy trade-off (Pareto frontier).

Worst at 100K/day

Timeouts truncate the reflexive loop's correction cycles: 0.871 F1, below every other design.

For A2A: every delegation hop adds cost, latency and a timeout to manage. The protocol is only half the design.

Demonstration: A2A + MCP in Action



A2A Layer: Discovery and Delegation

<> salesperson/client.py • A2A client

```
# Step 1: Discovery - fetch Agent Card, learn what the Intern can do
agent_card = await resolver.get_agent_card() # GET /.well-known/agent-card.json
# → name: "Intern", skills: ["search-products"]

# Step 2: Delegation - send a natural-language task (no SQL, no MCP)
message = Message(role=Role.user, parts=[Part(root=TextPart(text=user_text))])
async for event in client.send_message(message):
    task = event[0] if isinstance(event, tuple) else event
    if task.status:
        print(f"[Intern] task status -> {task.status.state}") # working ... completed
        for artifact in getattr(task, "artifacts", None) or []:
            final_text = artifact.parts[0].root.text # Step 3: collect Artifact
```

<> intern/executor.py • A2A server

```
class InternExecutor(AgentExecutor):
    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        task = context.current_task
        if not task:
            task = new_task(context.message)
            await event_queue.enqueue_event(task)

        updater = TaskUpdater(event_queue, task.id, task.context_id)
        await updater.start_work() # A2A: task → working

        user_text = context.get_user_input() # A2A: extract user message
        answer = await _run_agentic_loop(user_text) # MCP + LLM loop

        await updater.add_artifact(
            [new_text_artifact(name="search-results", text=answer).parts[0]],
            name="search-results",
        ) # A2A: return result
        await updater.complete() # A2A: task → completed
```

MCP Layer: Defining and Discovering Tools

<> catalog_server/server.py • provider side

```
@mcp.tool()
def query_products(max_price: float, category: Optional[str] = None) -> list[dict]:
    """Search the catalog for products at or below a given price,
    optionally filtered by category (electronics, stationery, home)."""
    ...

@mcp.tool()
def get_catalog_summary() -> dict:
    """Return aggregate statistics: categories, counts, and price ranges."""
    ...

@mcp.tool()
def compare_products(product_ids: list[int]) -> list[dict]:
    """Fetch multiple products by id for side-by-side comparison."""
    ...

...

if __name__ == "__main__":
    mcp.run(transport="streamable-http") # starts on http://127.0.0.1:8100/mcp
```

<> intern/executor.py • _run_agentic_loop() • consumer side

```
async with mcp_session() as session:
    tools = _mcp_tools_to_openai(await session.list_tools()) # Step 1: discover tools
    messages = [{"role": "system", ...}, {"role": "user", "content": user_text}]

    while True:
        choice = (await llm.chat.completions.create(model=MODEL,
                                                    tools=tools, messages=messages)).choices[0]

        if choice.finish_reason == "stop": # Step 4: final answer
            return choice.message.content

        for tc in choice.message.tool_calls: # Step 2: LLM chose a tool
            result = await session.call_tool( # Step 3: call via MCP
                tc.function.name,
                json.loads(tc.function.arguments)
            )
            messages.append({"role": "tool", "content": result}) # feedback → repeat
```

Some Examples

<> The Theory Examples ✨

```
./run_demo.sh "find me electronics under 20 euros"
./run_demo.sh "home products under 10 euros"
./run_demo.sh "tell me about the wireless mouse"
./run_demo.sh "compare the two cheapest electronics"

# query_products → compare_products
./run_demo.sh "find all electronics under 15 euros, compare them side by side,
and tell me which is the best value for money"

# get_catalog_summary → query_products (multiple categories)
./run_demo.sh "I have 30 euros to set up a home office. What should I buy across
all categories?"

# query_products × 3 (one per category)
./run_demo.sh "I want to buy one item from each category, all under 10 euros.
List the options per category and suggest the best combo"

# get_catalog_summary → query_products × N → synthesize
./run_demo.sh "I need 3 different gifts for colleagues, total budget 20 euros,
one per category. What do you recommend and why?"
```

<> The Painful Reality Examples 🤖

```
# wrong category name → LLM may pass category="office supplies" instead of
"stationery"
./run_demo.sh "show me office supplies under 5 euros"

# no price defined → LLM may invent max_price or skip tool call
./run_demo.sh "find me something cheap for my desk"

# ambiguous category → LLM may use wrong category or return empty
./run_demo.sh "any good deals on tech accessories?"

# hallucinated product → LLM may call get_product() with a made-up id
./run_demo.sh "tell me about the mechanical keyboard"

# unit mismatch → LLM may pass max_price=20 when user meant 20 dollars, not euros
./run_demo.sh "what electronics can I get for 20 bucks?"
```

Our Findings



Failures were semantic, not transport

Ambiguous requests led the LLM to pass a wrong filter to the tools. The protocols themselves never broke.



Layers changed independently

Changing the SQLite schema behind the MCP server required no change to the A2A contract with Agent A.

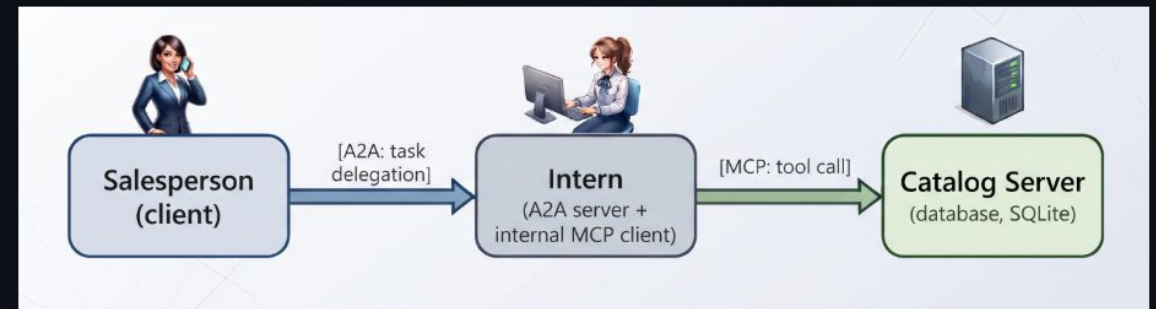


Hidden tools vs. discoverable skills

A new MCP server behind Agent B is invisible to Agent A: good for decoupling, but the new capability is discoverable only once declared as a skill in the Agent Card.

MCP + A2A - Demo Project

A working, end-to-end system that shows MCP and A2A together:



- **Catalog Server** (`catalog_server/`): exposes the product catalog (SQLite) as four MCP tools via Streamable HTTP.
- **Intern** (`intern/`): an A2A server that, when it receives a task, runs an LLM agentic loop that calls MCP tools to fulfill it.
- **Salesperson** (`salesperson/`): an A2A client that reads the Intern's Agent Card and delegates a natural-language task to it.

Summary & Core Takeaways



Two layers

MCP connects an agent to its tools; A2A connects agents to each other. Complementary, not competing.



Converging mechanics

Both now handle long tasks and mid-call input. What still differs is who is in control.



Capability limit

Standard interfaces don't fix reasoning: the best agents solve about half of realistic tasks.



Privacy by design

stdio is not a privacy guarantee, and opaque partners need contracts. Least privilege, always.

Thank You • Questions & Discussion

Open question: if MCP is now stateless and routed like any web API, what still makes it more than an API?

modelcontextprotocol.io • a2a-protocol.org • demo: [mcp_a2a_demo](#)